

Examen de la Convocatoria Ordinaria – 2021-2022

Tiempo disponible: 3 horas

Se quiere construir un programa en C++ que gestione el envío de los pedidos que los clientes realizan a través de la plataforma *JustLunch*, asignando cada envío a uno de los *riders* que tienen contratados. La empresa gestiona una lista de hasta `MaxPedidos` pedidos (`MaxPedidos` = 50) y tiene una lista de exactamente `NumRiders` riders (`NumRiders` = 8).

La información de los pedidos realizados está en el archivo `pedidos.txt`. Cada línea contiene los datos de un pedido, separados por un espacio: momento en el que se ha solicitado (compuesto por hora y minutos), tiempo en minutos que dura el envío, y si se trata de un pedido prioritario (1) o no (0). La última línea contiene -1 como horas (centinela).

```
13 0 17 1
13 0 14 0
13 0 16 0
13 0 22 1
13 3 19 0
13 5 17 1
13 7 18 0
...
13 59 10 1
-1
```

Además de los tres datos obtenidos del archivo (`momentoSolicitado`, `duración` y si es o no prioritario), mantendremos para cada pedido otros tres datos más: el momento (compuesto por hora y minutos) en que se inicia el envío del pedido (`momentoInicio`) que es el momento en el que el rider lo va a llevar, el número de rider al que se asigna el envío (valor de 0 a `NumRiders-1`) y si se ha procesado o no (es decir, si se le ha asignado un rider o no).

De cada rider se guarda el momento en la que está disponible para un nuevo pedido (inicialmente todos estarán disponibles a las 13:00 que es cuando empiezan a trabajar) y las ganancias acumuladas por los envíos que realiza (para lo que hay que tener en cuenta que se les paga, actualmente, 21 céntimos por cada minuto que dura un envío).

Detalles de implementación

En el programa se gestionan momentos (compuestos por horas y minutos), por lo que se define un tipo `struct tHora` que incluye las horas y los minutos de un momento en el tiempo. En la plantilla proporcionada se define ese tipo y se proporciona implementado el siguiente subprograma:

- `bool` `menor(tHora horaIzq, tHora horaDer)`: dados dos valores de tipo `tHora` devuelve `true` si la `horaIzq` es anterior a la `horaDer` y `false` en caso contrario.

Debes desarrollar los siguientes dos subprogramas relacionados con el tipo `tHora`:

- `void` `mostrar(tHora hora)`: dado un valor de tipo `tHora` lo muestra por pantalla con el formato HH:MM. [0,25 puntos]
- `tHora` `mas(tHora hora, int min)`: dados un valor de tipo `tHora` y un número de minutos, devuelve el valor de tipo `tHora` al que se llega tras sumar al valor `tHora` dado el número de minutos recibido. [0,25 puntos]

[1 punto] Define los tipos `tRider`, con los datos que se quieren tener de cada rider (ganancias y disponible, este último de tipo `tHora`), y `tListaRiders` para la lista de exactamente `NumRiders` riders.

[1 punto] Define también los tipos `tPedido` para los datos de cada pedido (`momentoSolicitado` de tipo `tHora`, `momentoInicio` de tipo `tHora`, `duracion`, `rider`, `prioritario` y `procesado`) y `tListaPedidos` para la lista de hasta `MaxPedidos` pedidos.

Implementa, al menos, los siguientes subprogramas:

- `carga()`: vuelca la información de `pedidos.txt` en la lista de pedidos. Devuelve `true` si se pudo abrir el archivo o `false` en otro caso. Si en el archivo hay más pedidos que el máximo permitido, se ignorarán los que sobren. **[1 punto]**
- `siguiente()`: dada la lista de pedidos devuelve la posición (índice), en la lista, del primer pedido prioritario no procesado. Si no quedan pedidos prioritarios sin procesar, devuelve el primero no procesado. Para implementar este subprograma puedes asumir que la lista recibida tiene algún pedido sin procesar. **[1,5 puntos]**
- `eligeRider()`: dada la lista de riders devuelve la posición (índice), en la lista, del rider que antes esté disponible. **[1 punto]**
- `planifica()`: dadas la lista de pedidos y la de riders, procesa todos los pedidos, empezando por los prioritarios y continuando por los demás, asignando a cada pedido el rider que antes esté disponible, rellenando los datos del pedido relativos al momento de inicio¹ y el rider asignado, así como marcándolo como procesado. También actualizará las ganancias del rider asignado² y su nueva disponibilidad³. Para cada pedido procesado además se mostrará por pantalla la asignación realizada, tal como se muestra en el ejemplo de ejecución (líneas sombreadas en gris)⁴. Para decidir en cada momento qué pedido toca procesar usarás el subprograma `siguiente`. **[2,5 puntos]**
- `muestra()`: dadas la lista de pedidos y la de riders, muestra para cada rider información sobre los pedidos que tiene asignados y sus ganancias (ver en el ejemplo de ejecución la información a mostrar, líneas sin sombrear). **[1 punto]**

El programa principal comenzará llamando a `carga()`. Si se han podido cargar los pedidos, llamará a `planifica()` para asignar un rider a cada pedido, y para terminar llamará a `muestra()` para ver los pedidos asignados a cada rider y sus ganancias. **[0,5 puntos]**

Recuerda que en el código que desarrolles se valorarán la corrección del diseño, la eficiencia del código, la ausencia de código repetido, la correcta utilización de los esquemas vistos en clase, etc. Recuerda que no se permite el uso de variables globales ni de instrucciones de salto, salvo `return` al final de las funciones y `break` en la instrucción `switch`.

Ejemplo de ejecución

¹ Si el momento en el que se ha solicitado el pedido es anterior al momento en el que el rider asignado está disponible, el momento en que se inicia el envío del pedido será el momento en el que esté disponible el rider; en caso contrario, será el momento en el que se solicita el pedido.

² La ganancia obtenida por servir un pedido se computa teniendo en cuenta los minutos que dura el pedido y lo pagado por minuto (21 cts).

³ La nueva disponibilidad del rider asignado a un pedido se computará sumando los minutos que dura el pedido a la hora de tipo `tHora` a la que se inicia el envío del pedido.

⁴ Se mostrará el momento en el que se ha solicitado, si es o no prioritario, la duración, el rider asignado, el momento en el que se inicia el envío, y el momento en el que será recibido.

Se asignan los riders...

Pedido a las 13:00	- Prioridad: S	- 17'	- Rider 1	- Envio: 13:00	-> 13:17
Pedido a las 13:00	- Prioridad: S	- 22'	- Rider 2	- Envio: 13:00	-> 13:22
Pedido a las 13:05	- Prioridad: S	- 17'	- Rider 3	- Envio: 13:05	-> 13:22
Pedido a las 13:10	- Prioridad: S	- 15'	- Rider 4	- Envio: 13:10	-> 13:25
Pedido a las 13:12	- Prioridad: S	- 16'	- Rider 5	- Envio: 13:12	-> 13:28
Pedido a las 13:16	- Prioridad: S	- 14'	- Rider 6	- Envio: 13:16	-> 13:30
Pedido a las 13:20	- Prioridad: S	- 19'	- Rider 7	- Envio: 13:20	-> 13:39
Pedido a las 13:20	- Prioridad: S	- 24'	- Rider 8	- Envio: 13:20	-> 13:44
Pedido a las 13:30	- Prioridad: S	- 14'	- Rider 1	- Envio: 13:30	-> 13:44
Pedido a las 13:45	- Prioridad: S	- 15'	- Rider 2	- Envio: 13:45	-> 14:00
Pedido a las 13:56	- Prioridad: S	- 14'	- Rider 3	- Envio: 13:56	-> 14:10
Pedido a las 13:59	- Prioridad: S	- 10'	- Rider 4	- Envio: 13:59	-> 14:09
Pedido a las 13:00	- Prioridad: N	- 14'	- Rider 5	- Envio: 13:28	-> 13:42
Pedido a las 13:00	- Prioridad: N	- 16'	- Rider 6	- Envio: 13:30	-> 13:46
Pedido a las 13:03	- Prioridad: N	- 19'	- Rider 7	- Envio: 13:39	-> 13:58
Pedido a las 13:07	- Prioridad: N	- 18'	- Rider 5	- Envio: 13:42	-> 14:00
Pedido a las 13:10	- Prioridad: N	- 16'	- Rider 1	- Envio: 13:44	-> 14:00
Pedido a las 13:15	- Prioridad: N	- 13'	- Rider 8	- Envio: 13:44	-> 13:57
Pedido a las 13:15	- Prioridad: N	- 17'	- Rider 6	- Envio: 13:46	-> 14:03
Pedido a las 13:15	- Prioridad: N	- 13'	- Rider 8	- Envio: 13:57	-> 14:10
Pedido a las 13:18	- Prioridad: N	- 18'	- Rider 7	- Envio: 13:58	-> 14:16
Pedido a las 13:20	- Prioridad: N	- 15'	- Rider 1	- Envio: 14:00	-> 14:15
Pedido a las 13:22	- Prioridad: N	- 12'	- Rider 2	- Envio: 14:00	-> 14:12
Pedido a las 13:25	- Prioridad: N	- 27'	- Rider 5	- Envio: 14:00	-> 14:27
Pedido a las 13:30	- Prioridad: N	- 11'	- Rider 6	- Envio: 14:03	-> 14:14
Pedido a las 13:40	- Prioridad: N	- 15'	- Rider 4	- Envio: 14:09	-> 14:24
Pedido a las 13:45	- Prioridad: N	- 13'	- Rider 3	- Envio: 14:10	-> 14:23
Pedido a las 13:50	- Prioridad: N	- 13'	- Rider 8	- Envio: 14:10	-> 14:23
Pedido a las 13:50	- Prioridad: N	- 8'	- Rider 2	- Envio: 14:12	-> 14:20
Pedido a las 13:55	- Prioridad: N	- 21'	- Rider 6	- Envio: 14:14	-> 14:35
Pedido a las 13:57	- Prioridad: N	- 11'	- Rider 1	- Envio: 14:15	-> 14:26
Pedido a las 13:59	- Prioridad: N	- 17'	- Rider 7	- Envio: 14:16	-> 14:33

Rider	Solicitado	Iniciado	Terminado
1	13:00	13:00	13:17
1	13:10	13:44	14:00
1	13:20	14:00	14:15
1	13:30	13:30	13:44
1	13:57	14:15	14:26
Ganancias: 15.33			
2	13:00	13:00	13:22
2	13:22	14:00	14:12
2	13:45	13:45	14:00
2	13:50	14:12	14:20
Ganancias: 11.97			
3	13:05	13:05	13:22
3	13:45	14:10	14:23
3	13:56	13:56	14:10
Ganancias: 9.24			
4	13:10	13:10	13:25
4	13:40	14:09	14:24
4	13:59	13:59	14:09
Ganancias: 8.40			
5	13:00	13:28	13:42
5	13:07	13:42	14:00
5	13:12	13:12	13:28
5	13:25	14:00	14:27
Ganancias: 15.75			
6	13:00	13:30	13:46
6	13:15	13:46	14:03
6	13:16	13:16	13:30
6	13:30	14:03	14:14
6	13:55	14:14	14:35
Ganancias: 16.59			
7	13:03	13:39	13:58
7	13:18	13:58	14:16
7	13:20	13:20	13:39
7	13:59	14:16	14:33
Ganancias: 15.33			
8	13:15	13:44	13:57
8	13:15	13:57	14:10
8	13:20	13:20	13:44
8	13:50	14:10	14:23
Ganancias: 13.23			

